Sprawozdanie z zajęć laboratoryjnych: Wielowarstwowy Perceptron (MLP)

Jan Taran

20 kwietnia 2026

Spis treści

1 Wstęp **2**

2 Opis implementacji **2**
2.1 Struktura sieci: . . . 2
2.2 Funkcje aktywacji . . . 2
2.3 Algorytm uczenia . . . 3
2.4 Optymalizatory . . . 3
2.4.1 Stochastic Gradient Descent (SGD) . . . 3
2.4.2 SGD + Moment (Momentum) . . . 3
2.4.3 RMSPROP . . . 4
2.4.4 ADAM (Adaptive Moment Estimation) . . . 4
2.5 Inicjalizacja wag . . . 4
2.6 Metody regularyzacji i wczesne zatrzymywanie . . . 5
2.6.1 Penalizacja `l1` i `l2` . . . 5
2.6.2 `Dropout` . . . 6
2.6.3 `Early stopping` . . . 6
2.7 Metoda `train` . . . 6

3 Wyniki eksperymentów i analiza **7**
3.1 Weryfikacja bazowej implementacji . . . 7
3.2 Analiza procesu uczenia . . . 8
3.2.1 Wybór optymalizatora . . . 8
3.2.2 Wybór funkcji aktywacji . . . 10
3.2.3 Wpływ rozmiaru paczki danych (Batch size) na jakość uczenia . . . 11
3.2.4 Wpływ inicjalizacji wag na stabilność uczenia . . . 13
3.3 Przeuczenie i regularyzacja . . . 13
3.3.1 Sprawdzenie działania penalizacji `l1` i `l2` . . . 14
3.3.2 Sprawdzenie działania `dropout` . . . 14
3.3.3 `Early stopping` . . . 15

4 Wnioski **16**

1 Wstęp

Celem niniejszego sprawozdania jest analiza działania i procesu uczenia sieci neuronowej typu *feedforward* (Multi-Layer Perceptron). Implementacja została wykonana od podstaw przy użyciu biblioteki *NumPy*, użytej tylko w celu szybszego mnożenia macierzy i funkcji losujących. Raport dokumentuje wpływ poszczególnych hiperparametrów na zdolności predykcyjne modelu w zadaniach regresji oraz klasyfikacji.

2 Opis implementacji

2.1 Struktura sieci:

Zdecydowałem się na implementacje macierzową, jej główną zaletą jest szybkość działania sieci. Stworzona została w tym celu tylko jedna klasa: MyNeuralNetwork. Użytkownik przy inicjalizacji musi podać jedynie liczbę neuronów na wejściu i może zmienić domyślne wartości liczby neuronów w warstwach ukrytych oraz w warstwie wyjściowej. Warstwy ukryte podawane są jako lista z liczbami całkowitymi, a warstwa wejściowa i wyjściowa jako liczby całkowite. Użytkownik przy inicjalizacji może dodatkowo podać funkcję aktywacji dla warstw ukrytych i oddzielnie dla warstwy wyjściowej. Może skorzystać z predefiniowanych z klasy ActivationFunctions lub stworzyć swoją własną, musi ona jedynie dziedziczyć po klasie ActivationFunction. Nie ma możliwości ustawienia różnych funkcji aktywacji w ramach warstw ukrytych.

Wagi są przechowywane w liście macierzy. Zastosowana została analogia do regresji liniowej, tzn. wagi dla i-tego wejścia znajdują się w i-tym wierszu, przez co aktywacja wygląda następująco: $x^T \cdot W$, gdzie $x \in R^n$ oraz W ma n wierszy i n_out kolumn, co jest równe ilości neuronów w następnej warstwie.

Takie właśnie podejście pozwala wykorzystać szybkie mnożenie macierzy z minimalną liczbą pętli, co przyspiesza działanie sieci.

```
def forward(self, data):
    activations = data.copy()
    for w in range(len(self.weights)):
        weighted_sum = np.dot(activations, self.weights[w]) +
                        self.biases[w]
        if w != len(self.weights) - 1:
            activations = self.activation_function.function(
                              weighted_sum)
        else:
            activations = self.last_activation_function.function(
                              weighted_sum)

    return activations
```

Listing 1: Zoptymalizowana metoda forward pass

2.2 Funkcje aktywacji

Zaimplementowane zostały następujące funkcje: Sigmoid, Linear, Tanh, ReLU oraz Softmax (tylko warstwa wyjściowa).

2.3 Algorytm uczenia

Zaimplementowana została metoda `back_propagation`, która aktualizuje wagi. Możliwy jest wybór optymalizatora i odpowiednich parametrów np. `learning_rate` jak również są opcje regularyzacji jak `l1`, `l2` lub `dropout`.

2.4 Optymalizatory

Zostały zaimplementowane następujące optymalizatory. Dla zachowania czytelności wzorów matematycznych przyjęto uniwersalne oznaczenia, w pełni odpowiadające zmiennym zaimplementowanym w kodzie źródłowym sieci:

- θ_t – aktualizowany parametr w bieżącym kroku t (reprezentujący wagi W lub biasy b),
- g_t – uśredniony gradient funkcji straty z pojedynczego mini-batcha (`grad_w` lub `grad_b`),
- η – współczynnik uczenia (`learning_rate`),
- β_1, β_2 – współczynniki zanikania (`beta1`, `beta2`),
- ϵ – stała stabilizująca, zapobiegająca dzieleniu przez zero (`epsilon`),
- m_t, v_t – wektory akumulujące wartości gradientów z poprzednich epok.

2.4.1 Stochastic Gradient Descent (SGD)

Podstawowy algorytm optymalizacji. Aktualizacja parametrów polega na przesunięciu ich wartości w kierunku przeciwnym do wektora gradientu, proporcjonalnie do ustalonego współczynnika uczenia η. Zaimplementowana metoda wyraża się elementarnym wzorem:

$$\theta_t = \theta_{t-1} - \eta \cdot g_t$$

2.4.2 SGD + Moment (Momentum)

Rozszerzenie klasycznego algorytmu SGD o element bezwładności (tzw. "pęd", zmienna m_t). Algorytm ten akumuluje kierunki z poprzednich kroków, co pozwala na szybsze pokonywanie płaskich rejonów funkcji straty i tłumienie niepożądanych oscylacji. W przygotowanej implementacji pęd jest obliczany jako suma aktualnego gradientu oraz pędu z poprzedniego kroku, przeskalowanego przez współczynnik β_1:

$$m_t = g_t + \beta_1 \cdot m_{t-1}$$

Następnie zaktualizowany pęd służy do przesunięcia parametrów modelu:

$$\theta_t = \theta_{t-1} - \eta \cdot m_t$$

2.4.3 RMSPROP

Algorytm z rodziny metod o adaptacyjnym współczynniku uczenia. W odróżnieniu od SGD, oblicza on wykładniczą średnią ruchomą kwadratów gradientów (v_t). Dzięki temu rozwiązaniu parametry, które rzadko ulegają dużym zmianom, zyskują efektywnie wyższy współczynnik uczenia, podczas gdy te mocno i często aktualizowane są wygaszane. Zgodnie z kodem, operacje te opisują wzory:

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot g_t$$

2.4.4 ADAM (Adaptive Moment Estimation)

Adam stanowi optymalne połączenie mechanizmów algorytmów Momentum oraz RMSProp. Algorytm śledzi zarówno wykładniczo zanikającą średnią z samych gradientów (pierwszy moment m_t), jak i z ich kwadratów (drugi moment v_t). Zgodnie z implementacją:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

Dodatkowo zaimplementowano krytyczny mechanizm korekty obciążenia (*bias correction*), który zapobiega dążeniu momentów (zainicjalizowanych jako wektory zerowe) do zera w początkowych fazach uczenia. Prawidłowe momenty w kroku t obliczane są następująco:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Ostateczna aktualizacja parametrów wykorzystuje oba skorygowane momenty:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

2.5 Inicjalizacja wag

Początkowe wartości wag mają bardzo duży wpływ na zbieżność algorytmów optymalizacyjnych oraz na zjawisko zanikania lub eksplozji gradientu. Aby zbadać ten wpływ, zaimplementowano trzy metody inicjalizacji macierzy wag W oraz wektorów biasów b. W poniższych wzorach n_{in} oznacza liczbę wejść do danej warstwy (liczbę wierszy macierzy wag), a n_{out} oznacza liczbę neuronów w tej warstwie (liczbę kolumn).

- **Rozkład jednostajny (Uniform):** Najprostsza metoda polegająca na losowaniu wartości z rozkładu równomiernego z przedziału $[0, 1)$. Wartości wag oraz biasów nie są w żaden sposób skalowane wielkością warstwy, co w głębokich sieciach może prowadzić do problemów numerycznych.

$$W, b \sim \mathcal{U}(0, 1)$$

- **Inicjalizacja He:** Metoda zaprojektowana specjalnie dla sieci wykorzystujących funkcję aktywacji ReLU (lub jej warianty). Wartości losowane są z rozkładu normalnego o średniej $\mu = 0$ oraz odchyleniu standardowym skalowanym w zależności od liczby wejść do warstwy. Pozwala to na utrzymanie wariancji sygnału na stałym poziomie podczas propagacji w przód. W zaimplementowanej wersji zarówno wagi, jak i biasy są losowane zgodnie ze wzorem:

$$W, b \sim \mathcal{N}\left(0, \sigma^2\right), \quad \text{gdzie} \quad \sigma = \sqrt{\frac{2}{n_{in}}}$$

- **Inicjalizacja Xaviera / Glorota (Xavier Normal):** Metoda optymalna dla symetrycznych funkcji aktywacji, takich jak Sigmoid czy Tanh. Jej celem jest utrzymanie stałej wariancji aktywacji oraz gradientów poprzez uwzględnienie w mianowniku zarówno liczby wejść (n_{in}), jak i wyjść (n_{out}) z warstwy. Zgodnie z kodem, odchylenie standardowe dla wag i biasów wynosi:

$$W, b \sim \mathcal{N}\left(0, \sigma^2\right), \quad \text{gdzie} \quad \sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

2.6 Metody regularyzacji i wczesne zatrzymywanie

Aby zapobiec zjawisku przeuczenia (overfittingu), czyli sytuacji, w której model traci zdolność do generalizacji na rzecz "uczenia się na pamięć" danych treningowych, w implementacji sieci zastosowano trzy komplementarne mechanizmy regularyzacyjne.

2.6.1 Penalizacja `l1` i `l2`

Kary L_1 (Lasso) oraz L_2 (Ridge) polegają na modyfikacji funkcji straty poprzez dodanie członu penalizującego duże wartości wag. Wpływa to bezpośrednio na proces propagacji wstecznej, wymuszając na optymalizatorze poszukiwanie rozwiązań opartych na mniejszych, bardziej zrównoważonych parametrach.

W zaimplementowanym modelu, całkowity błąd dla epoki jest powiększany o sumę kar ze wszystkich warstw, co można zapisać wzorem (gdzie λ_1 to `l1_lambda`, a λ_2 to `l2_lambda`):

$$L_{total} = L_0 + \lambda_1 \sum |W| + \lambda_2 \sum W^2$$

Ponieważ sieć operuje na optymalizacji z wykorzystaniem średniego błędu (dzielenie przez całkowitą liczbę próbek `n_samples` równe N), odpowiednio zmodyfikowano również wektory gradientów w kroku propagacji wstecznej. Gradienty dla macierzy wag przyjmują postać:

- Dla regularyzacji L_1: $\Delta W_{L1} = \frac{\lambda_1}{N} \cdot \text{sign}(W)$
- Dla regularyzacji L_2: $\Delta W_{L2} = \frac{2\lambda_2}{N} \cdot W$

Te dodatkowe wartości są bezpośrednio sumowane z wyliczonym analitycznie gradientem funkcji straty (`grad_w`).

2.6.2 `Dropout`

Dropout jest techniką architektoniczną polegającą na losowym wyłączaniu ułamka neuronów podczas każdego przejścia w przód, co zapobiega współzależnościom między neuronami (tzw. ko-adaptacji).

W modelu zaimplementowano **Inverted Dropout**. Polega on na wygenerowaniu maski z rozkładu dwumianowego, gdzie prawdopodobieństwo zachowania neuronu wynosi $1 - p$ (gdzie p to `dropout_rate`). Co kluczowe, aktywacje zachowanych neuronów są natychmiast skalowane w górę przez czynnik $\frac{1}{1-p}$:

$$M \sim \text{Binomial}(1, 1 - p)$$

$$a_{masked} = (a \odot M) \cdot \frac{1}{1 - p}$$

Dzięki temu zabiegowi wartość oczekiwana sumy wejść do następnej warstwy pozostaje stała. Eliminuje to konieczność modyfikowania wag podczas inferencji (testowania) podczas wyliczania błędów walidacyjnych zbiór przechodzi przez sieć bez użycia masek (`dropout_masks=None`). Wygenerowane maski są również wykorzystywane w procesie `back_propagation`, aby zablokować przepływ gradientu (wyzerować go) przez neurony, które były nieaktywne w danym mini-batchu.

2.6.3 `Early stopping`

W każdej epoce, po aktualizacji wag na zbiorze treningowym, sieć dokonuje predykcji na wydzielonym zbiorze walidacyjnym (`X_val`, `y_val`).

Zaimplementowany mechanizm posiada dwa parametry kontrolne:

- `min_delta` – próg określający, jaka minimalna różnica między bieżącym a najlepszym dotychczasowym błędem uznawana jest za faktyczną poprawę modelu,
- `patience` (*cierpliwość*) – liczba epok, przez które algorytm pozwala na kontynuowanie treningu pomimo braku spadku błędu walidacyjnego o co najmniej `min_delta`.

Gdy licznik braku poprawy (`patience_counter`) przekroczy zadaną wartość tolerancji, główna pętla uczenia zostaje przerwana. Sieć nie zachowuje jednak wag z ostatniej (potencjalnie przeuczonej) epoki. Zamiast tego uruchamiana jest procedura przywracania stanu – algorytm wczytuje z pamięci macierze wag (`best_weights`) oraz biasów (`best_biases`) pochodzące dokładnie z tej epoki, dla której błąd na zbiorze walidacyjnym osiągnął globalne minimum.

2.7 Metoda `train`

Aby ułatwić i zautomatyzować proces uczenia, zaimplementowano metodę `train`. Przeprowadza ona główną pętlę optymalizacyjną, obsługuje podział na mini-batche, aplikuje techniki regularyzacji oraz ostatecznie zwraca historię wartości funkcji straty (treningowej i walidacyjnej) oraz pełną historię zmian wag.

Metoda przyjmuje następujące parametry:

- `X_train, y_train` - Dane treningowe (cechy wejściowe i etykiety).
- `epochs` - Liczba pełnych przejść algorytmu przez zbiór treningowy.

- `learning_rate` - Współczynnik uczenia (η), określający rozmiar kroku aktualizacji wag.
- `weights_initiation_method` - Sposób inicjalizacji wag (`'uniform'`, `'he'`, `'xavier'` oraz `None` wtedy wagi nie są aktualizowane (przydatne jeśli chcemy kontynuować trening po wczytaniu wag)).
- `batch_size` - Rozmiar porcji danych (*mini-batch*) do jednorazowej aktualizacji gradientu.
- `optimizer` - Wybrany algorytm optymalizacji (`'sgd'`, `'momentum'`, `'rmsprop'`, `'adam'`).
- `save_weights` - Zapisywanie historii wag w trakcie uczenia (do późniejszej wizualizacji).
- `verbose` - Wypisywanie informacji o postępach treningu w konsoli.
- `early_stopping` - Przerwanie uczenia i przywrócenie najlepszych wag, gdy błąd walidacyjny przestaje maleć (chroni przed przeuczeniem).
- `X_val, y_val` - Zbiór walidacyjny (wymagany przy włączonym `early_stopping`).
- `patience` - Liczba epok oczekiwania na poprawę błędu walidacyjnego przed wczesnym zatrzymaniem.
- `min_delta` - Minimalna zmiana funkcji straty uznawana za faktyczną poprawę modelu.
- `l1, l2` - Aktywacja regularyzacji wag (L_1 – Lasso, L_2 – Ridge).
- `l1_lambda, l2_lambda` - Siła karania dużych wag (λ) dla poszczególnych regularyzacji.
- `dropout` - Aktywacja mechanizmu *Dropout* (losowe wygaszanie neuronów).
- `dropout_rate` - Prawdopodobieństwo wyłączenia pojedynczego neuronu w warstwie (np. 0.2).

3 Wyniki eksperymentów i analiza

3.1 Weryfikacja bazowej implementacji

Porównanie wyników własnej implementacji (forward pass) z wagami zaimportowanymi z sieci z biblioteki PyTorch na dwóch zbiorach i trzech różnych architekturach.

Sieć neuronowa w PyTorch'u była trenowana na 10000 epokach używając ADAM z $lr = 0.01$. Obie sieci używają sigmoida jako funkcji aktywacji. Dane były standaryzowane. MSE było liczone na danych niestandaryzowanych.

Porównanie przeprowadzono na dwóch zbiorach danych (`square-simple` oraz `steps-large`) dla trzech różnych architektur:

- Jedna warstwa ukryta, 5 neuronów (architektura 1-5-1),

- Jedna warstwa ukryta, 10 neuronów (architektura 1-10-1),
- Dwie warstwy ukryte, po 5 neuronów (architektura 1-5-5-1).

Wyniki błędu średniokwadratowego (MSE) zestawiono w Tabeli 1.

Tabela 1: Porównanie wartości błędu MSE na zbiorze testowym dla własnej implementacji oraz biblioteki PyTorch w celu weryfikacji propagacji w przód.

Zbiór	**Architektura**	**MSE (PyTorch)**	**MSE (Własny)**	**Różnica bezwgl.**
square-simple	1-5-1	0.8267955	0.8267883	0.0000072
	1-10-1	0.0223159	0.0223149	0.0000010
	1-5-5-1	0.0409658	0.0409663	0.0000005
steps-large	1-5-1	75.5577698	75.5577901	0.0000203
	1-10-1	66.4814224	66.4814502	0.0000278
	1-5-5-1	8.9240332	8.9240370	0.0000038

Dla przypadku o największej bezwzględnej różnicy (zbiór steps-large, architektura 1-10-1), różnica wynosi ≈ 0.0000278. Błąd względny wynosi zaledwie $\frac{0.0000278}{66.48} \approx 4.18 \cdot 10^{-7}$, czyli 0.000041%. Stanowi to potwierdzenie pełnej poprawności metody `feed_forward`.

3.2 Analiza procesu uczenia

3.2.1 Wybór optymalizatora

W celu rzetelnego porównania zaimplementowanych optymalizatorów przeprowadzono obszerny eksperyment oparty na przeszukiwaniu siatki hiperparametrów. Dla każdego z sześciu zbiorów danych oraz czterech optymalizatorów wyłoniono najlepszy wynik wygenerowany na podstawie różnych kombinacji trzech architektur sieci (o układach warstw ukrytych: $[5]$, $[5, 10, 5]$ oraz $[15, 25, 15]$) i czterech wartości współczynnika uczenia (`learning_rate`: 0.001, 0.005, 0.01 oraz 0.03). Każda testowana konfiguracja była trenowana przez 3000 epok z wykorzystaniem sigmoidalnej funkcji aktywacji. Takie podejście pozwoliło na wyeliminowanie przypadku i niezależne dobranie optymalnej struktury sieci dedykowanej dla każdego algorytmu uczącego.

Ostateczne wyniki zbadane na wydzielonych zbiorach testowych – wyrażone za pomocą metryki F-measure dla problemów klasyfikacji oraz błędu MSE dla zadań regresji – zostały szczegółowo zestawione w Tabelach 2 oraz 3.

Tabela 2: Najlepsze konfiguracje hiperparametrów i wyniki F-measure na zbiorze testowym dla zadań klasyfikacji.

Zbiór danych	**Optymalizator**	**Warstwy ukryte**	**Learning Rate**	**F-measure**
easy	adam	[5]	0.001	0.998000
easy	momentum	[5]	0.005	0.998000
easy	rmsprop	[5]	0.001	0.998000
easy	sgd	[5, 10, 5]	0.030	0.998000
rings3-regular	adam	[15, 25, 15]	0.001	0.973835
rings3-regular	momentum	[15, 25, 15]	0.005	0.975215
rings3-regular	rmsprop	[5, 10, 5]	0.010	0.978855
rings3-regular	sgd	[15, 25, 15]	0.030	0.964589
xor3	adam	[15, 25, 15]	0.005	0.965174
xor3	momentum	[5, 10, 5]	0.030	0.975369
xor3	rmsprop	[5, 10, 5]	0.030	0.967243
xor3	sgd	[5]	0.030	0.782548

Tabela 3: Najlepsze konfiguracje hiperparametrów i wyniki błędu MSE na zbiorze testowym dla zadań regresji.

Zbiór danych	**Optymalizator**	**Warstwy ukryte**	**Learning Rate**	**MSE**
multimodal-small	adam	[15, 25, 15]	0.005	2.983698
multimodal-small	momentum	[15, 25, 15]	0.030	479.503165
multimodal-small	rmsprop	[15, 25, 15]	0.005	1.678548
multimodal-small	sgd	[5, 10, 5]	0.030	1635.965631
square-simple	adam	[15, 25, 15]	0.005	0.236797
square-simple	momentum	[15, 25, 15]	0.030	1.608248
square-simple	rmsprop	[5]	0.005	1.143253
square-simple	sgd	[5]	0.030	95.008595
steps-large	adam	[5, 10, 5]	0.030	3.244888
steps-large	momentum	[15, 25, 15]	0.030	10.437617
steps-large	rmsprop	[5, 10, 5]	0.010	2.173680
steps-large	sgd	[5, 10, 5]	0.030	124.377428

(**SGD**) osiąga satysfakcjonujące zbieżności głównie na prostszych zbiorach danych, wymagając przy tym zastosowania relatywnie wysokich wartości współczynnika uczenia. Wzbogacenie tego algorytmu o człon bezwładności (**SGD z Momentem**) znacząco poprawia jego stabilność. Zauważono jednak, że przy tej metodzie najkorzystniejsze okazuje się jednoczesne obniżenie `learning_rate`, co skutecznie zapobiega powstawaniu zbyt dużych oscylacji błędu.

Średnio najwyższą skuteczność wykazuje **RMSProp**. Z przeprowadzonych symulacji wynika, że jego działanie jest najbardziej stabilne przy zastosowaniu architektur o średniej złożoności oraz niskich lub umiarkowanych wartościach współczynnika uczenia. Wysoce

konkurencyjne wyniki uzyskuje również algorytm **Adam**, będący koncepcyjnym połączeniem metod Momentum oraz RMSProp. Przez to jedna, że Adam wykorzystuje moment, algorytm ten wykazuje niekiedy tendencję do "przeskakiwania" (przestrzeliwania) wąskich minimów lokalnych funkcji straty. Efekt ten ma bezpośrednie odzwierciedlenie w nieznacznie słabszych rezultatach Adama na niektórych, bardziej specyficznych zbiorach danych.

3.2.2 Wybór funkcji aktywacji

Analogicznie do wyboru optymalizatora, podobnie przeprowadzono eksperyment na różnych funkcjach aktywacji. Dla każdego z sześciu zbiorów danych oraz trzech funkcji aktywacji wyłoniono najlepszy wynik wygenerowany na podstawie różnych kombinacji trzech architektur sieci (o układach warstw ukrytych: $[5]$, $[5, 10, 5]$ oraz $[15, 25, 15]$) i czterech wartości współczynnika uczenia (`learning_rate`: 0.001, 0.005, 0.01 oraz 0.03). Każda testowana konfiguracja była trenowana przez 10_000 epok z wykorzystaniem optymalizatora adam.

Wyniki testów zostały podzielone na problemy klasyfikacji (Tabela 4) oraz regresji (Tabela 5).

Tabela 4: Najlepsze konfiguracje i wyniki F-measure dla różnych funkcji aktywacji (klasyfikacja).

Zbiór danych	**Funkcja aktywacji**	**Warstwy ukryte**	**Learning Rate**	**F-measure**
easy	ReLU	[5]	0.001	0.998000
easy	Sigmoid	[5]	0.005	1.000000
easy	Tanh	[5]	0.010	0.998000
rings3-regular	ReLU	[15, 25, 15]	0.005	0.975689
rings3-regular	Sigmoid	[5, 10, 5]	0.001	0.984973
rings3-regular	Tanh	[15, 25, 15]	0.010	0.975667
xor3	ReLU	[5, 10, 5]	0.010	0.967283
xor3	Sigmoid	[15, 25, 15]	0.030	0.975369
xor3	Tanh	[5, 10, 5]	0.005	0.963194

Tabela 5: Najlepsze konfiguracje i wyniki błędu MSE dla różnych funkcji aktywacji (regresja).

Zbiór danych	**Funkcja aktywacji**	**Warstwy ukryte**	**Learning Rate**	**MSE**
multimodal-small	ReLU	[15, 25, 15]	0.005	3.770036
multimodal-small	Sigmoid	[5, 10, 5]	0.005	0.134649
multimodal-small	Tanh	[15, 25, 15]	0.001	0.618110
square-simple	ReLU	[15, 25, 15]	0.005	0.600356
square-simple	Sigmoid	[15, 25, 15]	0.001	0.047799
square-simple	Tanh	[5, 10, 5]	0.001	0.092170
steps-large	ReLU	[5, 10, 5]	0.010	2.927198
steps-large	Sigmoid	[5, 10, 5]	0.001	2.383209
steps-large	Tanh	[5, 10, 5]	0.001	2.356273

Analiza powyższych tabel prowadzi do interesujących wniosków, stojących częściowo w opozycji do współczesnych trendów w głębokim uczeniu maszynowym. W niemal każdym z testowanych przypadków (zarówno dla regresji, jak i klasyfikacji), to funkcja **Sigmoid** pozwoliła na osiągnięcie najlepszych ostatecznych rezultatów (najwyższe F-measure oraz najniższe błędy MSE). Wynik 1.0 dla zbioru `easy` oraz wybitnie niski błąd dla zbioru `multimodal-small` (0.134) potwierdzają doskonałe dopasowanie tej funkcji do charakterystyki dostarczonych danych.

Należy jednak podkreślić, że sukces funkcji Sigmoid w tym eksperymencie jest ściśle powiązany z długim procesem trenowania (10 000 epok) oraz użyciem optymalizatora Adam, który dzięki adaptacyjnym współczynnikom uczenia potrafił zniwelować problem zanikającego gradientu (*vanishing gradient*), typowego dla tej funkcji w głębokich sieciach. Funkcje **Tanh** oraz **ReLU** również poradziły sobie z zadaniami bardzo dobrze, przy czym ReLU wymagało zauważalnie wyższych wartości `learning_rate` (np. 0.010), aby skutecznie dostroić wagi.

3.2.3 Wpływ rozmiaru paczki danych (Batch size) na jakość uczenia

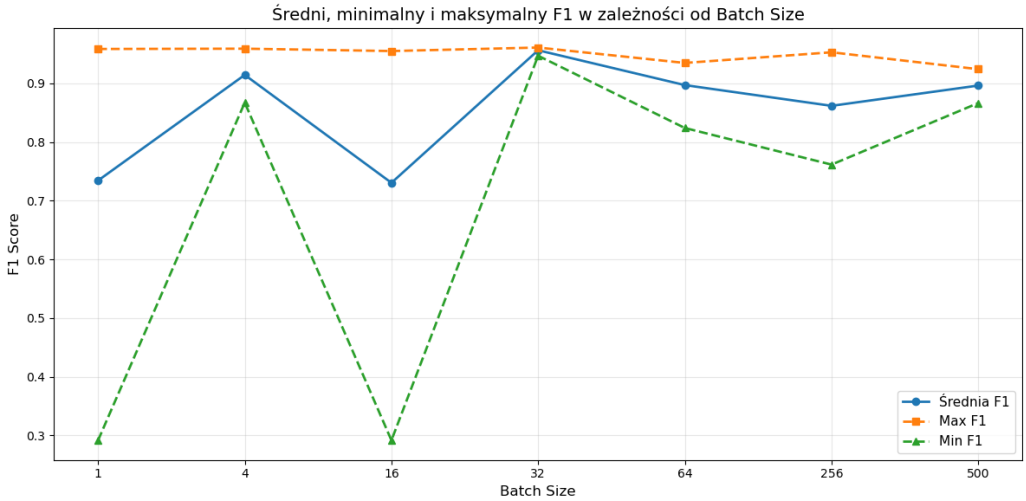
Kolejnym kluczowym hiperparametrem decydującym o dynamice procesu optymalizacji jest rozmiar paczki danych, na podstawie której wyliczany jest uśredniony gradient w pojedynczym kroku propagacji wstecznej. Aby zbadać jego wpływ na stabilność i końcową jakość klasyfikacji, przeprowadzono eksperyment na nieliniowym zbiorze danych `xor3`.

W celu izolacji wpływu badanego parametru, ustandaryzowano pozostałe elementy architektury:

- Warstwy ukryte: $[5, 10, 5]$
- Funkcja aktywacji: `ReLU`
- Inicjalizacja wag: Xavier
- Optymalizator: Adam (`learning_rate` = 0.005)
- Czas uczenia: 2500 epok

Przetestowano zbiór wartości `batch_size` $\in \{1, 4, 16, 32, 64, 256, N\}$, gdzie 1 oznacza czysty algorytm stochastyczny (aktualizacja po każdej próbce), natomiast N (w tym przypadku równe 500) to pełny spadek gradientowy (aktualizacja raz na epokę na podstawie całego zbioru). Ze względu na stochastyczny charakter procesu uczenia (losowa inicjalizacja wag oraz tasowanie próbek), dla każdego wariantu wykonano 3 niezależne próby, z których wyliczono wartość średnią, minimalną i maksymalną metryki F-measure (F1).

Zestawienie wyników numerycznych znajduje się w Tabeli 6, natomiast graficzną reprezentację stabilności uczenia w funkcji rozmiaru paczki ukazuje Rysunek 1.



Rysunek 1: Średnia, minimalna i maksymalna wartość metryki F1 na zbiorze testowym w zależności od rozmiaru paczki danych (skala logarytmiczna dla osi X).

Tabela 6: Statystyki metryki F-measure (F1) na zbiorze testowym dla różnych rozmiarów paczki danych (`batch_size`).

Batch Size	**Średnie F1**	**Minimalne F1**	**Maksymalne F1**
1	0.734441	0.291785	0.958598
4	0.914391	0.867150	0.959104
16	0.730559	0.291785	0.955068
32	0.956377	0.947078	0.961027
64	0.897055	0.823975	0.934794
256	0.861707	0.761579	0.952941
500 (Pełny zbiór)	0.896251	0.866434	0.924202

Zgodnie z teoretycznymi podstawami głębokiego uczenia mniejsze paczki danych wprowadzają znaczny szum w estymacie gradientu. Zjawisko to, choć spowalnia zbieżność i potrafi prowadzić do silnych oscylacji w trakcie treningu, często ułatwia ucieczkę z lokalnych minimów. Z kolei większe rozmiary `batch_size` gwarantują stabilny (gładki) kierunek spadku gradientu oraz szybsze obliczenia macierzowe, lecz stwarzają ryzyko przedwczesnej zbieżności do suboptymalnego minimum.

Jak można zaobserwować na wykresie, stabilność wyników (odstęp między wartością maksymalną a minimalną) oraz średnia wartość metryki F1 ulegają istotnym zmianom wraz z rozmiarem paczki. Najlepsze wyniki pod względem stabilności i najlepszego wyniku uzyskano na `batch_size` równym 32 co zgadza się z powszechną praktyką.

3.2.4 Wpływ inicjalizacji wag na stabilność uczenia

Początkowe wartości wag mają krytyczny wpływ na dynamikę procesu uczenia, zjawisko eksplodujących/zanikających gradientów oraz szanse na uniknięcie lokalnych minimów. Aby empirycznie zbadać ten wpływ, przeprowadzono analizę porównawczą na wymagającym zbiorze danych *multimodal-large*.

Dla zapewnienia spójności i porównywalności wyników, we wszystkich testach wykorzystano architekturę sieci z trzema warstwami ukrytymi o rozmiarach $[5, 10, 5]$. Ustalono hiperparametry procesu uczenia: optymalizator Adam, współczynnik uczenia $\eta = 0.005$, rozmiar paczki danych (*batch size*) równy 32 oraz liczbę epok 5000. Z uwagi na wysoce stochastyczny charakter inicjalizacji wag, eksperymenty dla każdej konfiguracji zostały powtórzone wielokrotnie. W Tabeli 7 zestawiono błąd minimalny, średni oraz maksymalny na zbiorze testowym.

Tabela 7: Zestawienie statystyk błędu MSE w zależności od funkcji aktywacji oraz metody inicjalizacji wag na zbiorze *multimodal-large*.

Funkcja aktywacji	**Inicjalizacja**	**Min MSE**	**Średnie MSE**	**Max MSE**
ReLU	he	45.0219	159.0297	266.9449
ReLU	uniform	81.1750	131.5869	258.2192
ReLU	xavier	74.1254	1163.0708	5291.0770
Sigmoid	he	2.4289	72.0286	264.0062
Sigmoid	uniform	3.6516	404.0493	1162.2361
Sigmoid	xavier	5.3181	395.9798	1109.0157
Tanh	he	0.9739	6.6201	25.5389
Tanh	uniform	0.8656	93.2846	518.3104
Tanh	xavier	2.9078	20.9544	67.1690

Przedstawione wyniki dobitnie ukazują, że skuteczność metody inicjalizacji jest nierozerwalnie związana z wybraną funkcją aktywacji, co stanowi potwierdzenie powszechnie uznanej teorii głębokiego uczenia maszynowego.

Zgodnie z matematycznymi założeniami, metoda Xaviera (zaprojektowana dla funkcji symetrycznych wokół zera) zastosowana w połączeniu z funkcją `ReLU` okazała się drastycznie niestabilna – maksymalny błąd MSE eksplodował do wartości przekraczających 5290. Z kolei w przypadku funkcji `Tanh`, zarówno inicjalizacja He, jak i Xaviera znacząco zredukowały średni błąd oraz wariancję w porównaniu do rozkładu jednostajnego (gdzie maksymalny błąd potrafił osiągnąć 518.31).

Zdecydowanie najlepszym dopasowaniem dla badanego zbioru *multimodal-large* okazała się funkcja `Tanh` z inicjalizacją He, zapewniająca wybitnie niski błąd minimalny (0.97) oraz największą stabilność i powtarzalność całego procesu uczenia (najniższy błąd średni rzędu 6.62).

3.3 Przeuczenie i regularyzacja

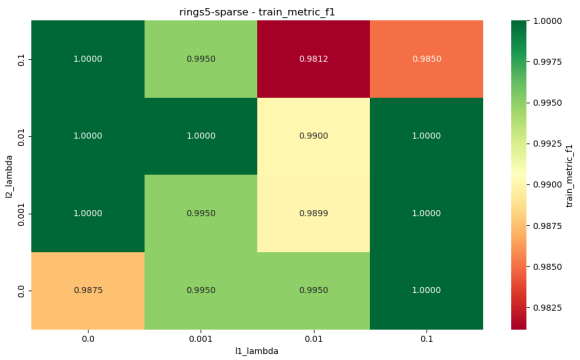
Zjawisko przeuczenia (overfittingu) jest jednym z głównych wyzwań podczas trenowania wielowarstwowych sieci perceptronowych, szczególnie w przypadku niewielkich lub rzadkich zbiorów danych.

Wdrożono i przetestowano trzy główne metody regularyzacji: penalizację wag (L_1 i L_2), wygaszanie neuronów (*Dropout*) oraz wczesne zatrzymywanie (*Early Stopping*).

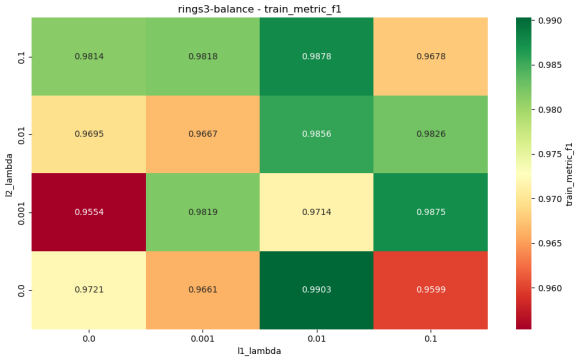
3.3.1 Sprawdzenie działania penalizacji `l1` i `l2`

Pierwszy etap eksperymentu polegał na analizie jednoczesnego wpływu regularyzacji Lasso (L_1) oraz Ridge (L_2). Metody te karzą model za zbyt duże wartości wag, co wymusza budowanie prostszych i bardziej zgeneralizowanych reprezentacji. Przeszukano siatkę parametrów dla pięciu wartości współczynników $\lambda \in \{0, 0.0001, 0.001, 0.01, 0.1\}$ zarówno dla kary L_1, jak i L_2.

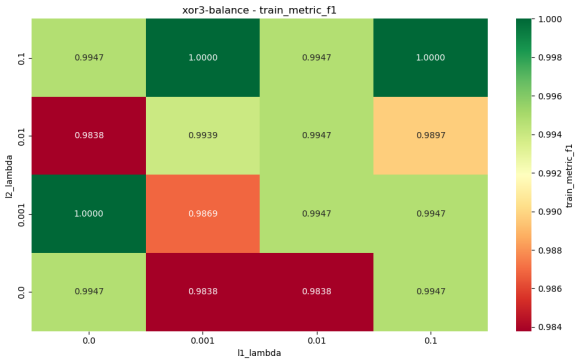
Na heatmapach 2 przedstawiono wartość błędu na zbiorze testowym w zależności od kombinacji obu współczynników dla każdego z czterech badanych zbiorów danych.



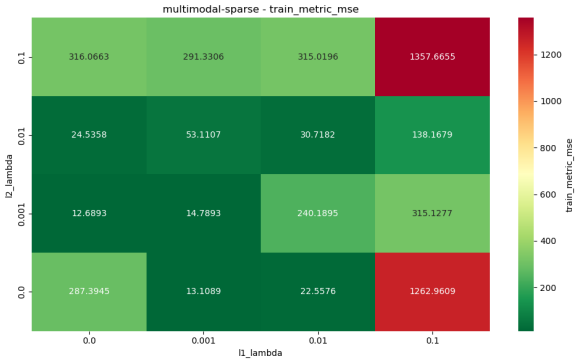
(a) `rings5-sparse`



(b) `rings3-balance`



(c) `xor3-balance`



(d) `multimodal-sparse`

Rysunek 2: Mapy ciepła przedstawiające skuteczność modelu w zależności od siły regularyzacji L_1 i L_2 na czterech zbiorach danych.

Na zbiorze z problem regresji widać, że duża penalizacja `l1` zdecydowanie pogarsza wyniki, ale równie dobrze widać, że niewielka penalizacja wpływa na MSE bardzo pozytywnie.

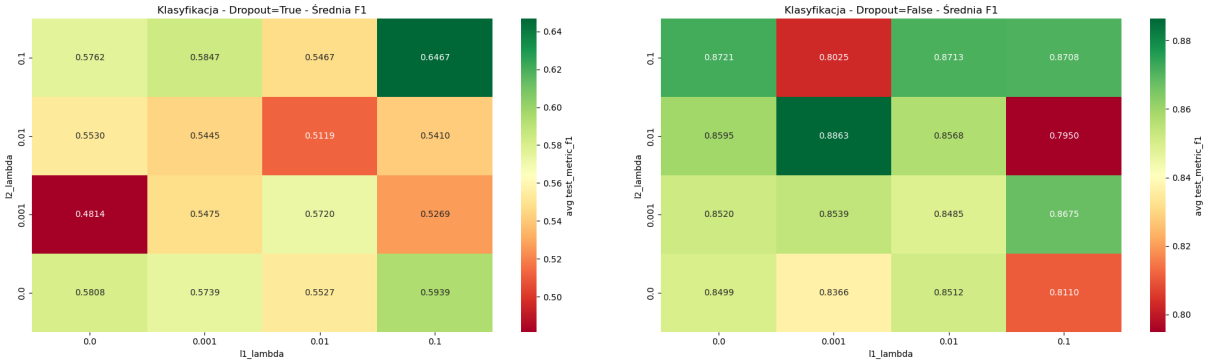
Na zbiorach z klasyfikacją najlepiej wybrać niską wartość współczynnika przy `l1` i raczej wyższe wartości przy penelizacji `l2`.

3.3.2 Sprawdzenie działania `dropout`

Kolejną testowaną techniką był mechanizm `dropout`, który zapobiega współzależnościom między neuronami poprzez ich losowe wyłączanie w trakcie treningu. W ramach ekspery-

mentu porównano warianty z wyłączonym oraz włączonym dropoutem.

Aby zniwelować wpływ specyfiki pojedynczego problemu, wyniki zostały uśrednione po trzech wybranych zbiorach danych. Rysunek 3 prezentuje zagregowane mapy ciepła pozwalające ocenić uśredniony wpływ tej metody na zdolność modelu do generalizacji wiedzy.



(a) Brak Dropoutu (`dropout=False`)

(b) Zastosowany Dropout (`dropout=True`)

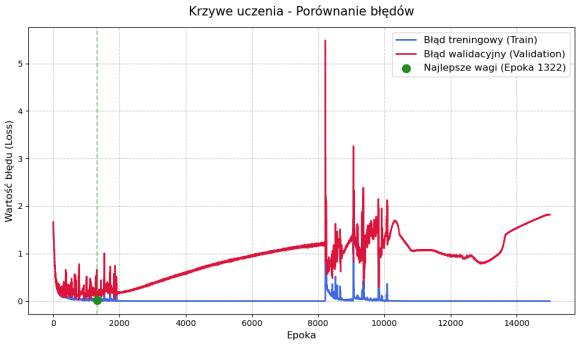
Rysunek 3: Porównanie uśrednionych wyników błędu testowego dla modeli trenowanych bez użycia i z użyciem mechanizmu Dropout.

Jak widać w tym przypadku `dropout` znacznie pogarsza wyniki, co jest tylko ciekawe to, że działa najlepiej z mocną regularyzacją. Prawdopodobnie nasze zbiory danych, gdzie używamy małych architektur i przewidujemy funkcje nie wymagają wygaszania neuronów.

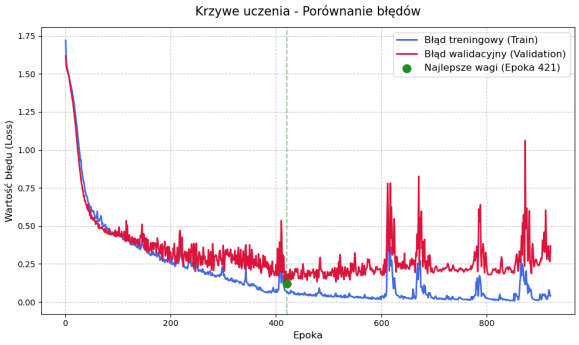
3.3.3 `Early stopping`

Ostatnim badanym mechanizmem było wczesne zatrzymywanie (`Early stopping`). Metoda ta polega na monitorowaniu wartości funkcji straty na niezależnym zbiorze walidacyjnym i przerwaniu procesu uczenia, gdy błąd ten zaczyna rosnąć, pomimo dalszego spadku błędu na zbiorze treningowym (co jest książkowym objawem przeuczenia).

Na wykresach poniżej zaprezentowano krzywe uczenia dla zbioru rings - klasyfikacja 4. Wyraźnie widać na nich moment, w którym sieć traci zdolność generalizacji. W większości przypadków wczesne zatrzymanie procesu uczenia sieci znacznie poprawia siłę predycji. Być może to niej jest najlepszy zbiór, ponieważ próbujemy przewidzieć funkcję bez losowego szumu, ale nadal widać poprawę przez małą liczność obserwacji w zbiorze treningowym.



(a) Krzywa uczenia bez wczesnego zatrzymywania – widoczny wzrost błędu walidacyjnego.



(b) Przerwanie treningu i przywrócenie wag przez `Early stopping`.

Rysunek 4: Wpływ mechanizmu wczesnego zatrzymywania na zjawisko przeuczenia widoczne na krzywych błędu.

Na tym zbiorze (multimodal-sparse) średnio MSE było poprawione z około 600 na 500 średnio na zbiorze testowym.

4 Wnioski

Przeprowadzone w ramach niniejszego sprawozdania eksperymenty, oparte na własnej implementacji prostej sieci neuronowej typu feed-forward, pozwalają na sformułowanie szeregu istotnych wniosków dotyczących wpływu hiperparametrów na proces uczenia sieci neuronowych.

Dobór funkcji aktywacji i inicjalizacja wag Z przeprowadzonych symulacji wynika, że dobór nieliniowości ma kluczowe znaczenie dla trudniejszych problemów, takich jak zbiory multimodalne. Wbrew współczesnym, powszechnym trendom (promującym funkcję ReLU), w testowanych środowiskach najwyższą skuteczność oraz najniższy błąd średniokwadratowy osiągała funkcja **Sigmoid** (oraz **Tanh** w połączeniu z inicjalizacją He). Należy jednak zaznaczyć, że sukces funkcji sigmoidalnych w tym przypadku był możliwy głównie dzięki zastosowaniu zaawansowanego optymalizatora (Adam) oraz długiego czasu uczenia, co skutecznie zniwelowało charakterystyczny dla nich problem zanikającego gradientu. Ponadto udowodniono, że błędne sparowanie inicjalizacji z funkcją aktywacji (np. metoda Xaviera z funkcją ReLU) może prowadzić do drastycznej niestabilności numerycznej i eksplozji błędu.

Wpływ algorytmów optymalizacyjnych na zbieżność Wybór optymalizatora diametralnie wpływał na dynamikę dochodzenia do minimum globalnego. Tradycyjny spadek gradientowy (SGD) radził sobie dobrze jedynie przy prostych zbiorach i wymagał manualnego, precyzyjnego doboru wyższego współczynnika uczenia. Wprowadzenie metod z adaptacyjnym krokiem uczenia, w szczególności **RMSProp** oraz **Adam**, pozwoliło na znacznie stabilniejsze uczenie sieci bez konieczności rygorystycznego dostrajania `learning_rate`. Mimo że Adam pozwalał na szybkie pokonywanie płaskich rejonów funkcji straty, na niektórych zbiorach jego człon pędu powodował sporadyczne "przestrzeliwanie" wąskich minimów, gdzie RMSProp okazywał się nieznacznie bardziej stabilny.

Znaczenie regularyzacji dla małych i rzadkich zbiorów danych Zbiory o niewielkiej liczbie próbek (typu *sparse*) wykazały ogromną podatność na zjawisko uczenia się na pamięć (overfittingu). Najbardziej niezawodnym mechanizmem obronnym okazało się **wczesne zatrzymywanie (*Early stopping*)**, które dynamicznie monitorowało zdolność do generalizacji i potrafiło odczuwalnie poprawić błąd na zbiorze testowym (np. spadek uśrednionego błędu MSE z 600 na 500 na zbiorze *multimodal-sparse*).

Penalizacja wag również okazała się użyteczna – delikatna kara L_1 wspierała modele regresyjne, natomiast dla problemów klasyfikacji skuteczniejsza była wyższa kara L_2 połączona z bardzo niskim L_1. Zbyt wysokie współczynniki λ natychmiast prowadziły jednak do silnego niedouczenia modelu (underfittingu). Co niezwykle interesujące, mechanizm **Dropout** w konfiguracji małych, rzadkich zbiorów danych przyniósł odwrotny skutek do zamierzonego – losowe wyłączanie neuronów w sytuacji, gdy sieć i tak dysponuje małą ilością informacji, prowadziło do zauważalnego pogorszenia wyników.

Podsumowując, eksperymenty dobitnie pokazały, że nie istnieje jedna, uniwersalna konfiguracja hiperparametrów w uczeniu głębokim. Skuteczność sieci neuronowej zależy od starannego, eksperymentalnego zestrojenia architektury, algorytmu optymalizacyjnego oraz metod regularyzacji z charakterystyką i wolumenem przetwarzanych danych.